Phala Network TEE 구현체 분석 - SideVM (격리 실행 환경)

1. SideVM 개요

SideVM은 Phala Network의 **격리된 가상머신 환경**으로, TEE 내에서 추가적인 보안 경계를 제공하여 복잡한 애플리케이션과 네이티브 코드를 안전하게 실행할 수 있게 합니다.

2. 핵심 구성 요소

2.1 SideVM 크레이트 구조

```
crates/sidevm/
├── host-runtime/ # SideVM 호스트 런타임
├── env/ # 실행 환경 추상화
├── macro/ # 매크로 지원
└── sidevm/ # 코어 SideVM 구현
```

2.2 주요 의존성 (host-runtime 기준)

```
# 가상화 및 실행
├── wasmer: WebAssembly 런타임
├── wasmer-compiler: WASM 컴파일러
```

```
├── wasmer-engine: 실행 엔진
├── wasmer-vm: 가상머신 구현

# 시스템 인터페이스
├── libc: C 라이브러리 바인딩
├── nix: Unix 시스템 콜 래퍼
├── mio: 비동기 I/O

# Phala 통합
├── phala-types: Phala 타입 정의
├── phala-crypto: 암호화 기능
└── pink: Pink 런타임 통합
```

3. 아키텍처 설계

3.1 격리 모델

```
TEE (SGX Enclave)
├── pRuntime
│   ├── Pink Runtime (스마트 컨트랙트)
│   └── SideVM Host
│   ├── SideVM Instance 1
│   ├── SideVM Instance 2
```

```
| └── SideVM Instance N
└── 공유 리소스 (제한적 접근)
```

3.2 보안 경계

- **TEE 경계**: SGX Enclave vs 호스트 OS
- **SideVM 경계**: SideVM vs pRuntime
- **인스턴스 경계**: SideVM 인스턴스 간 격리

4. 실행 환경 (sidevm-env)

4.1 환경 추상화

```rust
// 환경 인터페이스 예시 (추정)
trait SideVmEnvironment {
// 시스템 콜 인터페이스
fn sys_read(&mut self, fd: i32, buf: &mut [u8]) -> Result<usize>;
fn sys_write(&mut self, fd: i32, buf: &[u8]) -> Result<usize>;
fn sys_open(&mut self, path: &str, flags: i32) -> Result<i32>;
fn sys_close(&mut self, fd: i32) -> Result<()>;
// 메모리 관리
fn alloc_memory(&mut self, size: usize) -> Result<*mut u8>;
fn free_memory(&mut self, ptr: *mut u8, size: usize) -> Result<()>;
// 네트워킹
fn http_request(&mut self, req: HttpRequest) -> Result<HttpResponse>;
```

```
// 암호화
fn crypto_sign(&mut self, data: &[u8], key: &[u8]) -> Result<Vec<u8>>;
fn crypto_verify(&mut self, data: &[u8], sig: &[u8], key: &[u8]) -> Result<boo
}
```

4.2 시스템 콜 에뮬레이션

- **파일 시스템**: 가상 파일 시스템 제공
- **네트워킹**: 제한된 네트워크 접근
- **프로세스**: 프로세스 관리 에뮬레이션
- **메모리**: 안전한 메모리 할당/해제

5. 호스트 런타임 (host-runtime)

5.1 WASM 런타임 통합

```
// WASM 실행 예시 (추정)
struct SideVmInstance {
engine: wasmer::Engine,
module: wasmer::Module,
instance: wasmer::Instance,
memory: wasmer::Memory,
}
```

```rust
impl SideVmInstance {
fn new(wasm_code: &[u8]) -> Result<Self> {
let engine = wasmer::Engine::default();
let module = wasmer::Module::new(&engine, wasm_code)?;
let import_object = create_import_object(&engine)?;
let instance = wasmer::Instance::new(&module, &import_object)?;
let memory = instance.exports.get_memory("memory")?;
Ok(Self { engine, module, instance, memory })
}
fn call_function(&mut self, name: &str, args: &[wasmer::Value]) -> Result<Vec<
let func = self.instance.exports.get_function(name)?;
func.call(args)
}
}
```

5.2 리소스 관리

- **CPU 제한**: 실행 시간 제한
- **메모리 제한**: 메모리 사용량 제한
- **I/O 제한**: 입출력 대역폭 제한
- **네트워크 제한**: 네트워크 접근 제어

6. 매크로 시스템 (sidevm-macro)

6.1 코드 생성 매크로

```
// 매크로 사용 예시 (추정)
#[sidevm_export]
fn process_data(input: &[u8]) -> Vec<u8> {
// 복잡한 데이터 처리 로직
let processed = heavy_computation(input);
processed
}

#[sidevm_import]
extern "C" {
fn host_log(level: u32, message: *const u8, len: usize);
fn host_http_get(url: *const u8, url_len: usize) -> HttpResponse;
}
```

6.2 인터페이스 생성

- **Export 함수**: SideVM에서 호스트로 노출할 함수
- **Import 함수**: 호스트에서 SideVM으로 제공할 함수
- **타입 변환**: Rust 타입과 WASM 타입 간 변환
- **에러 처리**: 안전한 에러 전파

7. 보안 기능

7.1 샌드박싱

```
// 샌드박스 설정 예시 (추정)
struct SandboxConfig {
max_memory: usize,
max_execution_time: Duration,
allowed_syscalls: HashSet<SyscallType>,
network_policy: NetworkPolicy,
file_access_policy: FileAccessPolicy,
}

impl SandboxConfig {
fn  enforce(&self, vm: &mut SideVmInstance) -> Result<()> {
vm.set_memory_limit(self.max_memory)?;
vm.set_execution_timeout(self.max_execution_time)?;
vm.configure_syscall_filter(&self.allowed_syscalls)?;
vm.apply_network_policy(&self.network_policy)?;
vm.apply_file_policy(&self.file_access_policy)?;
Ok(())
}
}
```

7.2 격리 메커니즘

- **메모리 격리**: 각 인스턴스별 독립 메모리 공간
- **실행 격리**: CPU 시간 분할 및 제한
- **I/O 격리**: 파일 및 네트워크 접근 제어
- **상태 격리**: 인스턴스 간 상태 공유 방지

8. 성능 최적화

8.1 컴파일 최적화

```
// 컴파일러 설정 예시 (추정)
fn create_optimized_engine() -> wasmer::Engine {
let compiler = wasmer_compiler_cranelift::Cranelift::new();
let mut engine = wasmer::Engine::new(
Box::new(compiler),
wasmer::Target::default(),
wasmer::Features::default(),
);
// 최적화 설정
engine.set_optimization_level(wasmer::OptimizationLevel::Aggressive);
engine.enable_simd(true);
engine.enable_bulk_memory(true);
engine
}
```

8.2 실행 최적화

- **JIT 컴파일**: 런타임 최적화 컴파일
- **코드 캐싱**: 컴파일된 코드 재사용
- **메모리 풀링**: 메모리 할당 최적화
- **스레드 풀**: 스레드 재사용

9. 모니터링 및 디버깅

9.1 실행 추적

```
// 실행 추적 예시 (추정)
struct ExecutionTracer {
start_time: Instant,
memory_usage: usize,
syscall_count: HashMap<SyscallType, u64>,
execution_events: Vec<ExecutionEvent>,
}

impl ExecutionTracer {
fn trace_syscall(&mut self, syscall: SyscallType) {
*self.syscall_count.entry(syscall).or_insert(0) += 1;
self.execution_events.push(ExecutionEvent::Syscall(syscall));
}
fn trace_memory_alloc(&mut self, size: usize) {
```

```
self.memory_usage += size;

self.execution_events.push(ExecutionEvent::MemoryAlloc(size));

}

}
```

9.2 디버깅 도구

- **실행 로그**: 상세한 실행 로그
- **메모리 덤프**: 메모리 상태 덤프
- **성능 프로파일링**: CPU/메모리 사용량 분석
- **에러 추적**: 스택 트레이스 및 에러 정보

10. 사용 사례

10.1 복잡한 연산

- **머신러닝**: AI 모델 추론
- **이미지 처리**: 이미지 변환 및 분석
- **암호화**: 고급 암호화 알고리즘
- **데이터 분석**: 대용량 데이터 처리

10.2 레거시 코드 실행

- **C/C++ 라이브러리**: 기존 네이티브 라이브러리
- **Python 스크립트**: Python 인터프리터 실행
- **JavaScript**: V8 엔진 통합

- **기타 언어**: 다양한 언어 런타임 지원

이 문서는 SideVM의 격리 실행 환경 구현을 다룹니다. 다음 문서에서는 전체 시스템의 워크플로우와 상호작용을 분석합니다.